

Análisis y especificación del problema y los requerimientos funcionales

Cliente	Universidad ICESI / Empresa de Desarrollo de Software
Usuario	Estudiante / Jugador registrado
Contexto del problema	Se trata de una aplicación de escritorio que implementa una versión académica del juego Sokoban, donde el objetivo es mover cajas dentro de un tablero hasta ubicarlas en las posiciones correctas, y está pensada para que estudiantes puedan entender conceptos de programación, estructuras de datos y algoritmos mientras juegan. La interacción es por teclado y el estado del juego se visualiza en una interfaz gráfica. El sistema permite cargar niveles desde archivos externos, registrar información de las partidas, mostrar estadísticas del desempeño del jugador e incluso resolver niveles automáticamente mediante algoritmos de búsqueda. Todo esto garantizando que las reglas se respeten, que no se puedan hacer movimientos inválidos, que el progreso se guarde y que el jugador siempre tenga retroalimentación visual clara de lo que está pasando.
Requerimientos funcionales (listado)	• RF1 - Registrar jugador • RF2 - Guardar partida • RF3 - Cargar nivel • RF4 - Mover jugador • RF5 - Mostrar manual • RF6 - Empujar cajas • RF7 - Mostrar contadores • RF8 - Guardar al cerrar • RF9 - Cargar partida guardada • RF10 - Registrar estadísticas • RF11 - Mostrar ranking • RF12 - Deshacer movimiento • RF13 - Almacenar estados visitados • RF14 - Ejecutar solución automática
Requerimientos no funcionales (listado)	• RNF1 - Tecnología: El sistema debe desarrollarse como aplicación de escritorio en JavaFX • RNF2 - Arquitectura: El sistema debe seguir el patrón MVC (Model-View-Controller) • RNF3 - Rendimiento: El sistema debe responder a entradas del usuario en menos de 100ms • RNF4 - Confiabilidad: El sistema debe capturar todas las entradas del teclado sin pérdida • RNF5 - Persistencia: El sistema debe garantizar guardado correcto mediante Serializable • RNF6 - Testabilidad: El modelo debe ser probable independientemente de la interfaz gráfica • RNF7 - Mantenibilidad: El código debe ser modular con separación clara de responsabilidades

	• RNF8 - Calidad de código: El sistema debe aplicar principios de POO (encapsulamiento, herencia, polimorfismo) • RNF9 - Escalabilidad: El sistema debe soportar múltiples niveles cargados dinámicamente • RNF10 - Usabilidad: La interfaz debe ser intuitiva con retroalimentación visual clara
Requerimientos de proceso (listado)	• RP1 - El equipo debe estar conformado por máximo 3 integrantes • RP2 - El equipo debe trabajar de manera colaborativa • RP3 - El proyecto debe gestionarse usando Git y GitHub • RP4 - El repositorio debe tener estructura definida (README, doc, código) • RP5 - El equipo debe usar modelo de ramas Gitflow • RP6 - El equipo debe realizar al menos 15 commits representativos • RP7 - El equipo debe documentar métricas de calidad del software • RP8 - El equipo debe elaborar la especificación de requerimientos • RP9 - El equipo debe construir el diagrama de clases UML • RP10 - El equipo debe diseñar casos de prueba • RP11 - El equipo debe implementar pruebas unitarias automatizadas • RP12 - El equipo debe justificar estructuras de datos y algoritmos • RP13 - El equipo debe implementar estructuras de datos propias • RP14 - El código debe desarrollarse en inglés

Identificador y nombre	RF1 - Registrar jugador		
Resumen	El sistema debe permitir registrar un jugador ingresando nombre, correo electrónico, username, avatar y nivel de experiencia, validando los datos de entrada y almacenando la información para su uso en el juego.		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
	Nombre	String	No vacío
	Correo	String	Formato válido de correo
	Username	String	Único, no vacío
	Avatar	String	Selección válida
	Nivel experiencia	String	{Principiante, Avanzado, Experto}
Resultado o Postcondición	El jugador queda registrado en el sistema.		
Salidas	Nombre salida	Tipo de dato	Formato
	Confirmación registro	Boolean/String	Mensaje de éxito o error

Identificador y nombre	RF2 - Guardar partida		
Resumen	El sistema debe permitir guardar el estado actual de la partida, incluyendo la posición del jugador, las cajas y el tablero, para su posterior recuperación.		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos

	Estado del juego	Objeto	No nulo
	Nombre partida	String	No vacío
Resultado o Postcondición	La partida queda almacenada correctamente en el sistema.		
Salidas	Nombre salida	Tipo de dato	Formato
	Confirmación guardado	Boolean/String	Mensaje de éxito o error

Identificador y nombre	RF3 - Cargar nivel		
Resumen	El sistema debe permitir cargar niveles desde archivos externos en formato JSON, validando su estructura antes de representarlos en el tablero.		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
	Archivo nivel	Archivo JSON	Estructura válida
	Ruta archivo	String	Ruta existente
Resultado o Postcondición	El nivel queda cargado correctamente en el sistema.		
Salidas	Nombre salida	Tipo de dato	Formato
	Tablero cargado	Matriz	Representación bidimensional

Identificador y nombre	RF4 - Mover jugador		
Resumen	El sistema debe permitir mover el jugador en las cuatro direcciones posibles, validando colisiones con muros y otros objetos antes de ejecutar el movimiento.		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
	Dirección	String	{Arriba, Abajo, Izquierda, Derecha}
	Estado tablero	Matriz	Válido
Resultado o Postcondición	El jugador cambia de posición si el movimiento es válido.		
Salidas	Nombre salida	Tipo de dato	Formato
	Nueva posición jugador	Coordenadas	(x,y)
	Estado tablero actualizado	Matriz	Actualizado

Identificador y nombre	RF5 - Mostrar manual		
Resumen	El sistema debe permitir al usuario visualizar un manual o tutorial que explique las reglas del juego y su funcionamiento básico.		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos

	Solicitud de ayuda	Boolean	true
Resultado o Postcondición	El usuario puede visualizar el contenido del manual o tutorial.		
Salidas	Nombre salida	Tipo de dato	Formato
	Contenido del manual	String	Texto explicativo

Identificador y nombre	RF6 - Empujar cajas		
Resumen	El sistema debe permitir empujar cajas únicamente si la casilla siguiente está libre y verificar si todas las cajas están en posiciones objetivo para determinar la victoria.		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
	Posición jugador	Coordenadas	Válida
	Estado tablero	Matriz	Válida
Resultado o Postcondición	La caja se mueve correctamente y se evalúa la condición de victoria.		
Salidas	Nombre salida	Tipo de dato	Formato
	Estado tablero actualizado	Matriz	Actualizado
	Estado de victoria	Boolean	true/false

Identificador y nombre	RF7 - Mostrar contadores		
Resumen	El sistema debe mostrar en tiempo real los contadores de la partida, incluyendo número de movimientos, empujes realizados, nivel actual y tiempo transcurrido.		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
	Movimientos	Integer	≥ 0
	Empujes	Integer	≥ 0
	Nivel actual	Integer	≥ 1
	Tiempo	Integer	≥ 0
Resultado o Postcondición	Los contadores se visualizan correctamente en la interfaz del juego.		
Salidas	Nombre salida	Tipo de dato	Formato
	Contadores de juego	Enteros	Valores numéricos en pantalla

Identificador y nombre	RF8 - Guardar partida al cerrar el juego		
Resumen	El sistema debe guardar automáticamente el estado actual de la partida cuando el usuario cierra el juego.		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
	Evento de cierre	Boolean	true
	Estado del juego	Objeto	Válido
Resultado o Postcondición	La partida queda almacenada automáticamente antes de cerrar el sistema.		
Salidas	Nombre salida	Tipo de dato	Formato
	Confirmación de guardado	Boolean/String	Mensaje de éxito o error

Identificador y nombre	RF9 - Cargar partida guardada		
Resumen	El sistema debe permitir al usuario cargar una partida previamente guardada, restaurando el estado del juego.		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
	Archivo de partida	Archivo	Formato válido
	Selección de partida	String	Existente
Resultado o Postcondición	El estado del juego se restaura correctamente.		
Salidas	Nombre salida	Tipo de dato	Formato
	Estado del juego	Objeto	Restaurado

Identificador y nombre	RF10 - Registrar estadísticas		
Resumen	.El sistema debe registrar las estadísticas de cada partida del jugador, incluyendo movimientos, empujes realizados y tiempo empleado.		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
	Movimientos	Integer	>=0

	Empujes	Integer	≥ 0
	Tiempo	Integer	≥ 0
	Jugador	Objeto	Válido
Resultado o Postcondición	Las estadísticas quedan almacenadas asociadas al jugador.		
Salidas	Nombre salida	Tipo de dato	Formato
	Registro de estadísticas	Objeto	Guardado

Identificador y nombre	RF11 - Mostrar ranking		
Resumen	.El sistema debe generar y mostrar un ranking de jugadores basado en su desempeño en las partidas.		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
	Lista de estadísticas	Lista	No vacía
Resultado o Postcondición	Se genera un ranking ordenado de jugadores.		
Salidas	Nombre salida	Tipo de dato	Formato
	Ranking de jugadores	Lista	Ordenada según desempeño

Identificador y nombre	RF12 - Deshacer movimiento		
Resumen	.El sistema debe permitir al usuario deshacer la última acción realizada, restaurando el estado anterior del tablero.		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
	Historial de movimientos	Lista	No vacía
	Solicitud undo	Boolean	True
Resultado o Postcondición	El estado del juego regresa al estado anterior.		
Salidas	Nombre salida	Tipo de dato	Formato
	Estado del tablero	Matriz	Restaurado

Identificador y nombre	RF13 - Almacenar estados visitados		
Resumen	.El sistema debe almacenar los estados del tablero que ya han sido explorados durante la ejecución del juego o de algoritmos de búsqueda, con el fin de evitar ciclos y optimizar la exploración de soluciones.		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos

	Estado del tablero	Matriz	Válido
	Identificador de estado	String/Hash	Único
Resultado o Postcondición	El estado del tablero queda registrado en la estructura de control de estados visitados.		
Salidas	Nombre salida	Tipo de dato	Formato
	Estructura de estados visitados	Conjunto/Hash	Actualizada

Identificador y nombre	RF14 - Ejecutar solución automática		
Resumen	El sistema debe generar y ejecutar automáticamente una secuencia de movimientos que resuelva el nivel actual para la dificultad experto., utilizando algoritmos de búsqueda como BFS o DFS, permitiendo además su visualización en el tablero.		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
	Estado inicial del tablero	Matriz	Válido
	Algoritmo de búsqueda	String	BFS, DFS
Resultado o Postcondición	Se genera una solución válida para el nivel y se ejecuta en el sistema.		
Salidas	Nombre salida	Tipo de dato	Formato
	Secuencia de movimientos	Lista	Ordenada
	Visualización de solución	Animación	Representación en pantalla